
CuraHarness: Online Context Curation for Automated Harness Optimization

Anonymous Author(s)

Affiliation

Address

email

Abstract

Automated harness optimization refines the code that wraps a language model into an agent through an iterative propose–evaluate loop. Recent systems such as Meta-Harness replace compressed textual feedback with a coding agent that inspects the full filesystem history of prior candidates, exposing raw code, traces, and diffs to the proposer. Full access, however, is not effective access. As the optimization horizon grows, the accumulated history dilutes the proposer’s attention and forces it to rediscover which past runs and files matter for the next edit. In our traces, expanding the exposed workspace multiplies the number of available files, yet the proposer still opens only a small, nearly fixed number per iteration. We cast this bottleneck as an *online context curation* problem: before each proposal, the optimizer must decide which prior iterations, trace details, and workspace files to make salient under a limited inspection budget. We introduce CuraHarness, a framework that treats this decision as a control variable in automated harness optimization. CuraHarness combines two composable curation strategies. An iteration-level strategy bounds and labels historical evidence: it begins with narrow best/worst references and widens exposure when optimization stalls. This strategy alone improves mean held-out performance across our memory comparisons. A file-level strategy then layers on top: it assigns delayed credit to files read by the proposer and prioritizes those whose inspection correlates with productive edits. The two strategies are not independent additive components. Both route the same bounded proposer attention, only at different granularities. Across three benchmarks spanning software engineering and long-horizon memory, CuraHarness improves over the uncurated full-context baseline in most (benchmark, proposer) settings. On the two memory benchmarks it opens lower-cost operating points on the cost–quality frontier; on SWE-bench Verified it instead trades a modest cost increase for a much stronger harness, raising full-set passrate from 0.440 to 0.640 over the source harness. These results indicate that proposer context, usually treated as a static prompt artifact, is itself an online optimization variable that shapes harness search efficiency.

1 Introduction

Agent *harnesses*—the code that wraps a large language model (LLM) into a working agent—largely determine agent performance. Two agents built on the same LLM but with different harnesses can differ by up to $6\times$ on the same benchmark [1]. This gap has motivated *automated harness optimization*: an outer loop in which an LLM-driven proposer iteratively suggests candidate agents, prompts, or memory components, and an evaluator scores each candidate to guide the next proposal [2, 3, 4, 5, 6].



Figure 1: Pipeline overview of CuraHarness. The final artwork will replace this reserved slot: baseline candidates seed the frontier, the controller selects an iteration budget and file hints, the proposer edits a sandboxed harness, and evaluation feeds back into the next curation decision.

Most prior approaches inherit the information bottleneck of text optimization [7, 4, 8]. Feedback from earlier runs collapses into scalar scores, short summaries, or the most recent candidate [3, 5, 9, 10, 11], removing the details needed to trace downstream failures back to earlier harness decisions [6]. Meta-Harness lifts this bottleneck by replacing the proposer with a coding agent equipped with developer tools and exposing the full revision history as a navigable filesystem [6]. The meta-agent can inspect raw code, execution traces, and diffs before proposing the next edit.

Full access, however, is not effective access. As the optimization horizon grows, the filesystem accumulates prior candidates, failed edits, traces, and summaries, and the proposer must repeatedly judge which subset of that history bears on the current failure. Empirically, scaling the exposed workspace does not scale the number of files read: the proposer continues to open roughly the same small number per iteration and only shifts *which* files it picks. The bottleneck is therefore attention placement rather than raw access. We reframe automated harness optimization as an *online context curation* problem: before each proposal, the optimizer must decide which prior iterations, trace details, and workspace files to make salient under the proposer’s limited inspection budget.

We introduce CuraHarness, a framework that treats this context choice as a control variable in automated harness optimization. CuraHarness sits between the optimization loop and the meta-agent and decides which historical and file-level evidence should guide each proposal. The framework supports two composable curation strategies. The iteration-level strategy fixes the historical scope: it starts from a small labelled set of high- and low-performing prior attempts and widens exposure when optimization stalls. This strategy alone suffices to improve mean held-out performance across our memory comparisons. The file-level strategy then reallocates the proposer’s fixed read budget within the exposed workspace, using delayed reward to prioritize files whose inspection correlates with productive edits.

This design does not require both strategies to be active. Both address the same inspection bottleneck, only at different granularities. Iteration-level curation changes which parts of the search history are available and how they are labelled; file-level curation changes where the proposer spends its bounded inspection effort inside the exposed workspace. The two axes therefore need not produce independent, monotonic, or uniformly positive effects. They are alternative and composable ways to move the optimization loop along a cost–quality frontier.

We evaluate CuraHarness on three benchmarks spanning agentic software engineering and long-horizon memory: SWE-bench [12], LongMemEval [13], and LoCoMo [14]. Adaptive context curation improves held-out performance in most (benchmark, proposer) settings. On the two memory benchmarks it opens lower-cost operating points on the cost–quality frontier. On SWE-bench Verified it instead trades a modest increase in tokens per proposal for a markedly stronger final harness: the strongest curated run reaches 0.640 passrate on the full verified set, against 0.440 for the source harness and 0.458 for the full-context optimized baseline. Mechanistic analyses show that the proposer reads roughly the same number of files per iteration; only the target distribution shifts. At the iteration level, CuraHarness pulls reference-history reads away from a recency-biased default toward policy-labelled best/worst iterations. At the file level, it shifts the cost–quality operating point of that fixed read budget. Managing what the proposer attends to is therefore a first-class design variable in automated harness optimization.

78 2 Related Work

79 **Automated harness optimization.** The closest line of work uses LLMs as outer-loop optimizers
 80 for prompts, agent programs, or harness code. OPRO [15] casts prompt engineering as black-box
 81 optimization with LLM-generated candidates; GEPA [5] refines prompts using reflective textual feed-
 82 back. ADAS [7] moves from prompts to agent programs, having an LLM propose new scaffolds from
 83 an archive of prior designs. Meta-Harness [6] is closest to our setting: it optimizes harness files with
 84 a coding agent that can inspect the full filesystem history of previous candidates. Related program-
 85 search systems such as FunSearch, AlphaEvolve, and OpenEvolve use LLMs as mutators under
 86 automated fitness functions; we discuss these broader connections in Appendix A.1. CuraHarness
 87 keeps this file-backed optimization interface but changes the information policy around it. Rather than
 88 exposing the entire accumulated workspace or compressing it into a fixed summary, CuraHarness
 89 chooses which prior iterations and file hints should be salient before each proposal.

90 **Context curation in iterative optimization.** Our motivation aligns with evidence that more context
 91 is not always better. Liu et al. [16] show that models use long contexts unevenly; CODEFILTER [17]
 92 demonstrates that filtering retrieved code context can outperform passing all retrieved chunks. Nie
 93 et al. [18] further show that iterative generative optimization is sensitive to design choices such as
 94 credit horizon and experience batching. CuraHarness carries these ideas into automated harness
 95 optimization by treating proposer attention as an online control variable: a progressive iteration policy
 96 manages history scope, and delayed utility estimates manage file emphasis.

97 3 Method

98 This section describes CuraHarness, our framework for online context curation in automated harness
 99 optimization. CuraHarness builds on the observation that prior outer-loop systems either expose the
 100 full optimization history as a monolithic filesystem [6] or compress it into lossy summaries [15, 9, 5].
 101 Neither static rule is well calibrated to the search trajectory: which prior artifacts matter depends on the
 102 proposer’s current iteration, its recent edits, and the regressions it has just observed. CuraHarness
 103 instead curates context through two composable strategies acting at different granularities. The
 104 iteration-level strategy controls the historical scope by selecting which prior rounds’ artifacts to
 105 expose. The file-level strategy then reallocates read effort within the exposed workspace, prioritizing
 106 files whose inspection has previously correlated with productive edits. The file-level strategy is
 107 therefore not a standalone replacement for iteration-level curation: on its own it does not decide
 108 which prior iterations, traces, or candidate histories should enter the evidence workspace.

109 3.1 Problem Statement

Objective. A harness is a stateful program that wraps a fixed language model M and determines
 what context M sees at each step. Following [6], for a harness H and task instance $x \sim X$, we
 sample a rollout $\tau \sim p_M(H, x)$ and score it with task-specific reward $r(\tau, x)$. Automated harness
 optimization seeks

$$H^* = \arg \max_H \mathbb{E}_{x \sim X, \tau \sim p_M(H, x)} r(\tau, x).$$

110 **Search loop with context curation.** A run of the outer loop produces a sequence of evaluated
 111 candidates $\{(H_i, E_i)\}_{i=1}^t$, where E_i bundles H_i ’s source code, scores, and execution traces. We call
 112 the accumulated archive $D_t = \{(H_i, E_i)\}_{i \leq t}$ the experience pool. At iteration t , the proposer $\mathcal{P}$
 113 generates a new candidate conditioned on a curated context $C_t \subseteq D_t$:

$$H_{t+1} \sim P(\cdot \mid C_t), \tag{1}$$

114 where $C_t = \mathcal{C}(D_t, s_t)$ comes from a context curation policy $\mathcal{C}$ that selects which subset of D_t to
 115 surface, and s_t denotes the proposer’s auxiliary state at iteration t (e.g., current frontier, recent
 116 regressions). After evaluation, $D_{t+1} = D_t \cup \{(H_{t+1}, E_{t+1})\}$. This formulation generalizes prior
 117 work: Lee et al. [6] corresponds to direct filesystem access ($C_t = D_t$), while Agrawal et al. [5] and
 118 Yang et al. [15] use static compression ($C_t = \phi(D_t)$ for fixed ϕ).

119 3.2 CuraHarness

120 CuraHarness is a closed-loop controller for sequential context selection. At each iteration t , the
 121 controller (i) maintains a state s_t summarizing past outcomes, (ii) computes scores Q_t for candidate
 122 historical artifacts and files in D_t , and (iii) applies a selection rule π that maps these scores to the
 123 curated context C_t in Eq. 1. After the evaluator returns E_{t+1} , the controller updates its state using a
 124 reward signal ρ before the next proposal. The rest of this section defines the shared state, scoring,
 125 selection, and update interface; the following subsections instantiate it as the Progressive and bandit
 126 policies.

127 **State.** The controller’s state has two components matching the two curation granularities, $s_t =$
 128 (U_t, B_t) . U_t is an *iteration-level summary* that tracks how recent proposals have fared against the
 129 Pareto frontier—for instance, whether the last iteration was an improvement, or how many iterations
 130 have passed without one. B_t holds *per-file usage statistics*: for each file in the workspace, how often
 131 it has been read in past iterations and the cumulative reward attributed to those reads.

132 **Score.** Given (D_t, s_t) , CuraHarness estimates the expected value of surfacing each artifact for the
 133 next proposal, weighed against the inspection cost it would impose.

134 **Selection.** The selection rule turns these scores into an evidence workspace for the proposer. De-
 135 pending on the instantiated strategy, this workspace may include labelled reference iterations, trace
 136 summaries, file-level hints, or any combination of these signals under a budget tier.

137 **Update.** Once the evaluator returns E_{t+1} , CuraHarness computes a scalar reward ρ_{t+1} summarizing
 138 whether the new harness improved the search, updates the Pareto frontier, and refreshes $s_t \rightarrow s_{t+1}$.
 139 The refreshed state shapes the next round’s scoring and selection.

140 We instantiate this framework as two concrete policies at different granularities: an *iteration-level*
 141 policy deciding *which prior rounds* to surface, and a *file-level* policy deciding *which files inside the*
 142 *exposed workspace* receive priority.

143 3.3 Iteration-level curation.

144 Not every past iteration is equally informative, and its value shifts with the proposer’s current
 145 trajectory. After a successful update that extends the Pareto frontier, the proposer benefits most from
 146 the recent winning candidates, which reinforce the direction that just worked. After a streak of failed
 147 attempts, the proposer needs a broader view: exposing earlier failures and their traces can surface
 148 overlooked failure modes and prompt qualitatively different edits.

149 We operationalize this intuition through a *budget tier* $b_t \in \{\text{low}, \text{medium}, \text{high}\}$ that controls how
 150 much of the experience pool D_t is surfaced to the proposer. Table 1 specifies the context exposed at
 151 each tier: at low, the proposer sees only the single best-scoring candidate, the single worst-scoring
 152 one, and the most recent evaluation trace; at high, it receives the full history.

Table 1: Budget tiers and the context they expose to the proposer. “Reference iters” determines which prior candidates are included; “Trace scope” controls how many recent per-task evaluation traces accompany each reference bundle.

	low	medium	high
Reference iters	best 1 + worst	best 3 + worst	full history
Trace scope	last 1	last 3	all

153 The tier follows a *progressive* schedule keyed to the reward signal ρ (Section 3). The budget starts
 154 at low. Whenever a proposal fails to improve on the current Pareto frontier ($\rho_{t+1} \leq 0$), the tier is
 155 promoted by one level, gradually widening the proposer’s view of the experience pool. When an
 156 improvement is observed ($\rho_{t+1} > 0$), the tier resets to low, since a lean context suffices to refine the
 157 direction that just worked. This schedule realizes a simple explore–exploit tradeoff: the controller
 158 stays at a narrow context by default and only widens exposure when progress stalls.

File-level curation.

1. Given archive D_t , frontier F_t , file stats B_t , and stagnation count g_t , choose budget b_t : `low` if no file stats, `high` if $g_t \geq 4$, otherwise `medium`.
2. Select reference iterations from D_t according to b_t .
3. For each readable non-core file, estimate utility from smoothed success rate and mean delayed reward; penalize large reads and add a UCB bonus.
4. Pin core harness files, mark top-scored files as `hot`, and mark the next group as `warm`.
5. Run the proposer with selected references, pinned files, and hot/warm hints.
6. Evaluate the candidate; attribute reward to files read by the proposer; update B_{t+1} and g_{t+1} .

Figure 2: Pseudocode for file-level curation.

159 **3.4 File-level Curation.**

160 The file-level strategy treats readable files as arms in a delayed-credit bandit. Its scope is deliberately
 161 narrower than the iteration policy: once an evidence workspace has been assembled, it ranks files
 162 inside that workspace so the proposer sees a short hot list rather than an undifferentiated tree.
 163 The policy records which files were read before each proposal, assigns those reads a reward after
 164 evaluation, and uses the resulting utility estimates to build the next hot workspace. Figure 2 gives the
 165 procedure in pseudocode.

166 The schedule is dynamic because the useful scope changes during search. Keeping every iteration
 167 at `low` is cheap but closes the reference-iteration channel exactly when later proposals need to
 168 understand earlier design changes and avoid repeating exhausted edits. `Medium` keeps a compact set
 169 of high-value references open; `high` is reserved for stagnation, when the proposer benefits from a
 170 wider view of previous candidates and traces. Once an improvement is found, the controller returns
 171 to narrower context, since the next edit is again anchored by a recent productive direction.

172 **4 Experiments**

173 We evaluate CuraHarness by separating the two context-curation strategies it introduces. The
 174 iteration-level strategy bounds and labels the history seen by the proposer; the file-level strategy then
 175 reallocates attention inside the exposed workspace. The two are not independent additive modules;
 176 both route the same bounded proposer attention, only at different granularities.

177 **4.1 Setup**

178 **Benchmarks.** We use two long-horizon memory benchmarks and one software-engineering bench-
 179 mark. LoCoMo [14] probes multi-session conversation memory; we optimize on 80 training examples
 180 and report on the full 1,449-example split. LongMemEval [13] probes retrieval, temporal reasoning,
 181 and consolidation over long histories; we optimize on 100 training examples and report on the 400-
 182 example test split. SWE-bench Verified [19] probes multi-file coding-agent harnesses; we optimize
 183 on `trainfirst30` and re-evaluate selected candidates on the full 500-issue verified set.

184 **Harnesses and proposers.** On the memory benchmarks the inner harness is a MemGPT-style memory
 185 agent [20]. On SWE-bench the inner solver is a fixed Mini-SWE agent driven by DeepSeek v4
 186 Flash, which isolates harness-curation effects from solver changes. We instantiate the outer proposer
 187 with two coding-agent scaffolds: a Codex CLI proposer with GPT-5.4 (*codex54*) and a Claude
 188 Code-compatible proposer with Kimi K2.6 (*claudekimi*). Both run on the memory benchmarks;
 189 SWE-bench uses *claudekimi* only.

190 **Baselines and reporting.** The end-to-end baseline is **Full-context**, the default Meta-Harness
 191 configuration [6]: every iteration exposes the full revision history and an unrestricted file tree, with no
 192 best/worst labelling. On LoCoMo and LongMemEval, each (benchmark, proposer, policy) cell is run
 193 three times for 30 propose-evaluate iterations. We report test mean $\pm$ std across runs, the best held-out
 194 passrate, and per-propose token cost on the best-by-test run. SWE-bench is reported separately, since
 195 it is a single 20-iteration run with a different cost profile.

Table 2: Memory benchmark results. Each row aggregates three 30-iteration runs. Cost is millions of tokens per propose on the best-by-test run; Δ is relative to the cell’s full-context row.

Cell	Policy	test mean $\pm$ std	best test	total/prop	Δ
<i>LoCoMo / claudekimi</i>					
	full-context	0.331 $\pm$ 0.015	0.342	3.42M	—
	Progressive	0.355 $\pm$ 0.021	0.373	1.86M	−46%
	CuraHarness	0.355 $\pm$ 0.008	0.362	2.63M	−23%
<i>LoCoMo / codex54</i>					
	full-context	0.331 $\pm$ 0.013	0.340	3.42M	—
	Progressive	0.374 $\pm$ 0.014	0.388	2.46M	−28%
	CuraHarness	0.355 $\pm$ 0.027	0.387	2.14M	−37%
<i>LongMemEval / claudekimi</i>					
	full-context	0.498 $\pm$ 0.030	0.530	3.71M	—
	Progressive	0.505 $\pm$ 0.013	0.520	3.37M	−9%
	CuraHarness	0.510 $\pm$ 0.038	0.545	2.55M	−31%
<i>LongMemEval / codex54</i>					
	full-context	0.497 $\pm$ 0.010	0.508	2.93M	—
	Progressive	0.499 $\pm$ 0.028	0.528	2.66M	−9%
	CuraHarness	0.463 $\pm$ 0.010	0.473	2.18M	−26%

4.2 Main Results

Table 2 reports the four memory cells. Progressive isolates the iteration-axis policy: it exposes best/worst-tagged reference iterations under the low $\rightarrow$ medium $\rightarrow$ high schedule without invoking file-level utility. CuraHarness layers the file-axis bandit on top of the same historical-scope controller.

Iteration-level curation improves search quality. Progressive lifts mean held-out passrate over full-context in all four memory cells: +2.3 pp on LoCoMo claudekimi, +4.3 pp on LoCoMo codex54, +0.7 pp on LongMemEval claudekimi, and +0.25 pp on LongMemEval codex54. The LoCoMo gains are clear relative to run variance; the LongMemEval gains are smaller and should be read as directional. The iteration-axis signal alone therefore improves search without any file-level utility model.

File-level curation improves the operating point. CuraHarness is cheaper than full-context in all four memory cells and the cheapest policy in three of them, strictly improving both cost and best-test quality over full-context in those three. The exception, LongMemEval/codex54, is an informative trade-off: CuraHarness reaches a −26% cost point at a −3.5 pp best-test quality drop. The file axis should therefore be read as Pareto-frontier expansion rather than a universal dominance guarantee.

4.3 SWE-bench with Mini-SWE

We report SWE-bench separately because this experiment departs from the memory protocol in three ways: it was run once, it used 20 optimization iterations rather than 30, and its best curated runs spent more tokens per proposal than full-context. Despite the higher cost, the verified-set gain is the largest effect we observe.

The SWE-bench result clarifies what curation is and is not optimizing. Curation is not simply a token-reduction mechanism: Progressive and CuraHarness spend more tokens than full-context here. The curated context instead induces a different search trajectory. The proposer appears to use

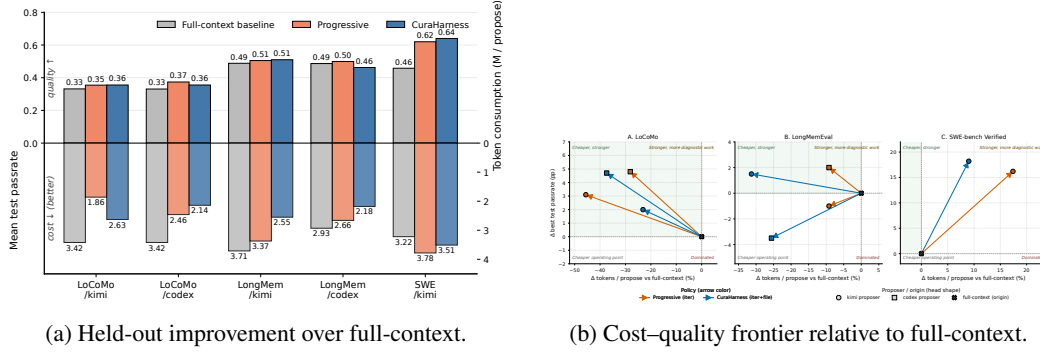


Figure 3: Optimization effect and frontier movement. Bars show test-passrate gains over the full-context baseline; the Pareto panel shows that CuraHarness introduces favorable cost-quality operating points rather than only shifting absolute quality.

Table 3: SWE-bench Verified with the Mini-SWE inner solver. This is a single 20-iteration claudekimi run; verified passrate is measured on the full 500-issue set. Cost is millions of tokens per propose.

Policy	trainfirst30	full verified	total/prop	verified Δ
source harness	—	0.440	—	—
full-context	0.500	0.458	3.22M	—
Progressive	0.533	0.620	3.78M	+16.2 pp
CuraHarness	0.533	0.640	3.51M	+18.2 pp

the narrower, labelled history to inspect more source and candidate evidence relevant to repair; this costs more tokens but yields a better harness on the full verified set.

4.4 Mechanism: labels redirect a fixed inspection budget

The introduction already noted that workspace size grows much faster than the proposer’s actual read count. Here we focus on where those reads land. For every Read call into `reference_iterations/iter_M/`, we classify whether iter M was the current top-1 or top-3 candidate by passrate at prompt-build time, and whether it sat among the three most recent prior iterations.

Progressive pulls reference reads toward the current best iteration: top-1 share rises by +25.6 pp on LongMemEval and +22.7 pp on LoCoMo. LoCoMo additionally shows a −23.1 pp drop in recent-3 share, so labels do not merely add attention to strong candidates; they can also pull attention away from the default recency prior.

This also explains why always staying at low is not enough. At low and medium, Progressive supplies compact best/worst labels useful for exploiting a known direction. As the run proceeds, the proposer must reason about more prior edits, regressions, and interactions among files. When progress stalls, opening medium or high gives the proposer enough historical evidence to grasp what has already changed and avoid locally plausible but redundant edits. The dynamic schedule treats low budget as a focus mechanism, not as a permanent information cap.

4.5 Ablation: how do the strategies interact?

We run two ablations to separate bounded historical exposure, labelled historical attention, and file-level utility guidance. First, on LoCoMo/codex54 we compare full-context against three bounded iteration-selection rules: Random@3, Recency@3, and Progressive. Second, on claudekimi memory runs we force the bandit budget to stay at either low or high.

Table 4: Reference-read target shares. Reads are bucketed by candidate rank or recency at prompt-build time and aggregated over two claudekimi runs per cell.

Benchmark	Policy	ref Reads	top-1	top-3	recent-3	other
LongMemEval	full-context	430	52.1%	82.3%	60.2%	3.7%
	Progressive	421	77.7%	87.6%	55.6%	5.2%
LoCoMo	full-context	485	31.1%	66.6%	59.4%	8.7%
	Progressive	422	53.8%	70.4%	36.3%	15.6%

Table 5: Ablations. Top block: LoCoMo/codex54 test results over three 30-iteration runs. Bottom block: claudekimi force-budget train passrate.

Cell / setting	Policy	score	best test	total/prop	ref share
<i>Iter-axis selection ablation (LoCoMo / codex54)</i>					
	full-context	0.331 $\pm$ 0.013	0.340	3.42M	—
	Random@3	0.360 $\pm$ 0.027	0.391	2.66M	—
	Recency@3	0.346 $\pm$ 0.035	0.386	2.72M	—
	Progressive	0.374 $\pm$ 0.014	0.388	2.46M	—
	CuraHarness	0.355 $\pm$ 0.027	0.387	2.14M	—
<i>Force-budget ablation (claudekimi memory, train)</i>					
LoCoMo	bandit force=high	0.4000	—	4.21M	36.4%
LoCoMo	bandit force=low	0.3875	—	2.79M	0.1%
LongMemEval	bandit force=high	0.5800	—	3.73M	36.4%
LongMemEval	bandit force=low	0.6300	—	2.83M	0.1%

242 All bounded variants beat full-context on LoCoMo/codex54, so part of the gain comes from avoiding
243 unstructured full-history browsing. The force-budget rows show why the dynamic policy still matters:
244 force-low nearly closes the reference-history channel, while force-high pays more tokens to keep
245 it open. LongMemEval carries enough file-level signal for force-low to do well on train, whereas
246 LoCoMo benefits from explicit historical examples. The policy must therefore expand when later
247 iterations need broader evidence, yet should not spend the whole run at high budget when a compact
248 context suffices.

249 5 Limitations

250 Our results support context curation as a design principle rather than universal dominance for any
251 single policy. The number of retained runs per cell is small, so we avoid significance claims on
252 individual cells and instead report run means, standard deviations, and best-by-test operating points.
253 The file-level strategy also shows benchmark- and proposer-dependent trade-offs: it often opens
254 lower-cost operating points but can sacrifice passrate when the uncurated file pool already carries
255 strong signal or when guidance metadata is layered on top of a large exposed history.

256 The proposer itself is a black-box coding-agent optimizer. Both Claude Code-compatible and
257 Codex CLI-compatible proposers carry model behaviour, system prompts, tool-use policies, and
258 implementation details that CuraHarness cannot fully control. Our policies bias the proposer’s
259 behaviour distribution by changing which iterations and files are salient, but they cannot force a
260 particular sequence of tool calls or guarantee that the proposer uses the provided hints. This matters
261 for cost interpretation. On SWE-bench, curated runs consume more tokens than full-context while
262 also reaching substantially higher verified-set passrate. The policy appears to redirect the proposer
263 toward more useful inspection, yet a more controllable proposer interface could expose a cleaner
264 trade-off between passrate and token budget.

The ablations are necessarily narrower than the main evaluation. The Random@3 and Recency@3 iteration-selection ablations are anchored on the LoCoMo / codex54 setting, while the force-budget analyses focus on retained claudekimi runs. The read-target mechanism analysis is most detailed for claudekimi because its tool traces expose per-file read targets; codex54 logs bucket more activity under a single shell interface, which limits comparable file-level attribution. Finally, CuraHarness is evaluated on three agent-harness families, but all experiments use a fixed outer-loop length and a fixed set of proposer backbones. Larger run budgets, additional proposers, and broader task families may shift the cost-quality frontier and should be tested before treating the observed trade-offs as deployment defaults.

6 Conclusion

We introduced CuraHarness, a framework for online context curation in automated harness optimization. Rather than treating a coding proposer’s filesystem access as an all-or-nothing choice, CuraHarness makes the proposer’s evidence workspace an explicit control variable. Its iteration-level strategy bounds and labels historical evidence, while its file-level strategy can be composed on top to reallocate read effort within the exposed workspace. Across memory and software-engineering harnesses, this framing improves held-out performance in most (benchmark, proposer) settings. On the two memory benchmarks it opens lower-cost operating points on the cost-quality frontier; on SWE-bench Verified it instead trades a modest cost increase for a substantially stronger curated harness. The results do not imply that more curation is always better, nor that the two strategies contribute independently. They show instead that the proposer’s limited inspection budget is itself part of the optimization problem, and that routing this budget deliberately can make harness search more effective.

References

- [1] Muxin Tian, Zhe Wang, Blair Yang, Zhenwei Tang, Kunlun Zhu, Honghua Dong, Hanchen Li, Xinni Xie, Guangjing Wang, and Jiaxuan You. Swe-bench mobile: Can large language model agents develop industry-level mobile applications? *arXiv preprint arXiv:2602.09540*, 2026.
- [2] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [3] Alexander Novikov, Ngân V u, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [4] Asankhaya Sharma. OpenEvolve: Open-source implementation of AlphaEvolve. <https://github.com/codelion/openevolve>, 2025.
- [5] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gega: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [6] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [7] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [8] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026.

- [9] Yoonho Lee, Joseph Boen, and Chelsea Finn. Feedback descent: Open-ended text optimization via pairwise comparison. *arXiv preprint arXiv:2511.07919*, 2025.
- [10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [11] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [13] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [14] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [15] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.
- [16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [17] Yanzhou Li, Shangqing Liu, Kangjie Chen, Tianwei Zhang, and Yang Liu. Impact-driven context filtering for cross-file code completion. In *Conference on Language Modeling*, 2025.
- [18] Allen Nie, Xavier Daull, Zhiyi Kuang, Abhinav Akkiraju, Anish Chaudhuri, Max Piasevoli, Ryan Rong, YuCheng Yuan, Prerit Choudhary, Shannon Xiao, Rasool Fakoor, Adith Swaminathan, and Ching-An Cheng. Understanding the challenges in iterative generative optimization with LLMs. *arXiv preprint arXiv:2603.23994*, 2026.
- [19] OpenAI. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, August 2024. Accessed: 2026-05-04.
- [20] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [21] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.
- [22] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.
- [23] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, Genetic and Evolutionary Computation, pages 331–366. Springer, 2023.
- [24] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

- [25] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [26] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [27] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [28] Xin He, Kaiyong Zhao, and Xiaowen Chu. AutoML: A survey of the state-of-the-art. *Knowledge-Based Systems*, 212:106622, 2021.

A Technical appendices and supplementary material

A.1 Supplementary related work

Prompt and context optimization. Beyond the optimization systems most directly related to CuraHarness, several methods search over prompts or agent context. EvoPrompt [21] couples evolutionary operators with LLM mutation, while DSPy [22] compiles declarative module signatures into optimized few-shot chains. ACE [8] evolves system prompts and memory playbooks through generation, reflection, and curation stages. These methods motivate treating context as an optimization target, but they do not address the file-backed, multi-iteration workspace exposed to coding-agent proposers.

Evolutionary and program search. FunSearch [2] pairs an LLM with island-based evolution to discover programs under automated fitness functions. AlphaEvolve [3] extends this style of search to multi-file codebases with a coding-agent proposer, and OpenEvolve [4] provides an open implementation. ELM [23] formalizes LLMs as general mutation operators within evolutionary computation. These systems motivate LLM-based program search, but they maintain populations or evolutionary selection pressure over programs. In contrast, CuraHarness optimizes agent harnesses, maintains a Pareto frontier of candidates, and studies how to curate the filesystem evidence read by a single coding-agent proposer.

LLM agents and technical foundations. Modern LLM agents such as MemGPT [20], Reflexion [24], Voyager [25], and SWE-agent [26] are optimization targets or benchmark scaffolds rather than competing optimizers in our experiments. On the algorithmic side, our file-level policy draws on UCB1 [27]: each workspace file is treated as an arm, reward is computed from recent harness performance, and an exploration bonus prevents premature convergence. More broadly, the work connects to AutoML [28], with a search space defined by natural-language code edits rather than a fixed configuration grammar.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer **[No]**, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or **[No]**, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.